

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesor:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: FAUSTO DAMIAN GUANO LOYA

PARALELO: A

SEMESTRE: QUINTO

ACTIVIDAD: Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

PERIODO FEBRERO 2026 - AGOSTO 2026



Índice

Actividad # 1 Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk).....	2
1.1 Descripción General del Sistema	2
1.2 Actores Principales.....	2
1.2.1 Usuario Final	2
1.2.2 Técnico de Soporte	2
1.2.3 Administrador del Sistema	3
1.3 Requerimientos Funcionales	3
1.4 Requerimientos No Funcionales	4
2. Modelado UML Estándar	5
2.1 Diagrama de Casos de Uso	5
2.2 Diagrama de Clases	5
3. Propuesta e Implementación de Patrones de Diseño	6
3.1 Patrón Creacional: Factory Method (Fábrica).....	6
3.2 Patrón de Comportamiento: State (Estado).....	6
3.3 Patrón de Comportamiento: Observer (Observador)	7
4. Flujo de Trabajo Profesional con Gitflow	7
5. Evidencias	7
5.1 Inicializar y estructura base:.....	7
5.2 Trabajar en el Análisis y Casos de Uso (feature/analisis-casos-uso):	8
5.3 Trabajar en el Diagrama de Clases y Patrones (feature/diagrama-clases-patrones):	8
5.4 Preparar la Entrega Final (release/v1.0.0):.....	9
6. Conclusiones	10
Bibliografía	12



Actividad # 1 Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

1.1 Descripción General del Sistema

El sistema de Help Desk para Data Center es una plataforma de gestión de incidentes técnicos diseñada para administrar y resolver fallos en infraestructura de servidores. Permite a los usuarios finales reportar incidentes de hardware, red y software; a los técnicos gestionar y resolver tickets asignados; y a los administradores supervisar el ciclo de vida completo de los incidentes.

1.2 Actores Principales

1.2.1 Usuario Final

Empleado o cliente que experimenta un fallo técnico y necesita reportarlo al sistema. Sus acciones principales incluyen:

- Crear y enviar tickets de incidentes describiendo el fallo.
- Consultar el estado de sus tickets activos.
- Confirmar la resolución de un incidente o solicitar reapertura.
- Recibir notificaciones automáticas sobre el progreso de su ticket.

1.2.2 Técnico de Soporte

Profesional especializado en algún área de infraestructura (red, hardware o software). Sus responsabilidades incluyen:

- Recibir y atender tickets asignados según su área de especialización.
- Actualizar el estado del ticket durante el proceso de resolución.
- Escalar incidentes que superen su nivel de competencia.
- Documentar la solución aplicada para generar base de conocimiento.



1.2.3 Administrador del Sistema

Responsable de la gestión global de la plataforma y de la operación del Data Center. Sus funciones son:

- Configurar categorías de incidentes, SLAs y reglas de asignación automática.
- Supervisar el panel de control con métricas en tiempo real.
- Asignar técnicos manualmente en casos críticos o de escalamiento.
- Generar reportes estadísticos de desempeño y disponibilidad.
- Administrar usuarios, roles y permisos del sistema.

1.3 Requerimientos Funcionales

ID	Requerimiento	Descripción
RF-01	Crear Ticket	El usuario final puede registrar un nuevo incidente indicando título, descripción, categoría (Hardware/Red/Software) y nivel de prioridad (Baja/Media/Alta/Crítica).
RF-02	Categorizar Fallo	El sistema clasifica automáticamente el incidente según palabras clave o el técnico puede reclasificarlo manualmente.
RF-03	Asignar Técnico	El motor de asignación selecciona el técnico más adecuado según su especialidad, carga de trabajo actual y disponibilidad.



RF-04	Gestionar Estados	Los tickets transitan por estados definidos: Abierto → En Progreso → Pendiente → Resuelto → Cerrado.
RF-05	Notificaciones	El sistema envía alertas automáticas por correo/SMS al crear un ticket, cambiar de estado, escalar o cerrar un incidente.
RF-06	Escalamiento	Si el tiempo de resolución supera el SLA definido, el sistema escala automáticamente el ticket al supervisor o técnico senior.
RF-07	Base de Conocimiento	El técnico documenta la solución aplicada; el sistema la almacena para consultas futuras de incidentes similares.
RF-08	Reportes y Estadísticas	El administrador accede a dashboards con métricas: tiempo promedio de resolución, tickets por categoría, productividad de técnicos.

1.4 Requerimientos No Funcionales

ID	Atributo de Calidad	Descripción
RNF-01	Escalabilidad	La arquitectura soporta crecimiento horizontal. El módulo de tickets puede desplegarse en múltiples instancias sin afectar la consistencia.



RNF-02	Mantenibilidad	El uso de patrones de diseño SOLID y Gitflow garantiza que el código sea comprensible, modificable y extensible con mínimo impacto.
RNF-03	Trazabilidad	Cada cambio de estado, asignación o modificación queda registrado con autor, fecha y hora para auditoría completa.
RNF-04	Disponibilidad	El sistema debe estar disponible 99.5% del tiempo (alta disponibilidad), con tiempos de respuesta menores a 2 segundos.
RNF-05	Seguridad	Control de acceso basado en roles (RBAC), cifrado de datos sensibles en tránsito y en reposo.

2. Modelado UML Estándar

Para los diagramas, puedes utilizar herramientas como Lucidchart, Draw.io o StarUML.

Asegúrate de cumplir con la notación formal:

2.1 Diagrama de Casos de Uso

Debe reflejar el comportamiento del sistema desde el punto de vista de los actores.

- Tips de diseño: Usa relaciones de <<include>> para pasos obligatorios (ej. Crear Ticket <<include>> Autenticar Usuario) y <<extend>> para flujos opcionales (ej. Resolver Ticket <<extend>> Escalar a Nivel 2).

2.2 Diagrama de Clases

Debe representar la estructura estática del sistema. Aquí ya debes empezar a vislumbrar



las clases que soportarán tus patrones de diseño.

- Clases core: Usuario, Ticket, HistorialIncidente, Categoría, Prioridad.
- Tips de diseño: Asegúrate de marcar correctamente la visibilidad de los atributos (privados -, públicos +) y los métodos. Define bien las relaciones de Agregación (un Ticket tiene un Historial) y Composición si aplica.

3. Propuesta e Implementación de Patrones de Diseño

La guía te pide al menos tres patrones de diseño (Creacionales, Estructurales o de Comportamiento). Para un sistema de Help Desk, estos tres encajan a la perfección y justifican su uso técnica y comercialmente:

3.1 Patrón Creacional: Factory Method (Fábrica)

- ¿Por qué usarlo?: Los incidentes pueden ser de diferentes tipos (ej. TicketHardware, TicketSoftware, TicketRedes). Cada uno podría requerir campos específicos, flujos de validación distintos o SLAs iniciales diferenciados.
- Solución: Creas una clase base o interfaz Ticket y una clase TicketFactory con un método crearTicket(tipo, datos). Así, el cliente del código no se preocupa por el new específico de cada objeto, desacoplando la creación de la lógica del negocio.

3.2 Patrón de Comportamiento: State (Estado)

- ¿Por qué usarlo?: Un ticket pasa por múltiples estados (Abierto, EnProgreso, Resuelto, Cerrado). El comportamiento del ticket cambia drásticamente según su estado (un ticket Cerrado no puede ser editado; un ticket Abierto no puede tener una solución registrada). Llenar el código de if/else o switch para validar el estado actual rompe el principio de Responsabilidad Única y Open/Closed.
- Solución: Defines una interfaz EstadoTicket con métodos como atender(),



resolver(), cerrar()). Luego, creas clases concretas para cada estado (EstadoAbierto, EstadoEnProgreso, etc.). La clase Ticket delega sus acciones al objeto de estado actual.

3.3 Patrón de Comportamiento: Observer (Observador)

- ¿Por qué usarlo?: Cada vez que un ticket cambia de estado o recibe un nuevo comentario, se debe notificar a múltiples interesados (al Cliente por correo, al Técnico por la app, y quizás al Administrador por un canal de auditoría).
- Solución: La clase Ticket actúa como el Sujeto (Subject). Las clases de notificación (EmailNotificationService, PushNotificationService) implementan la interfaz Observer. Cuando el ticket cambia, invoca a notificar(), actualizando automáticamente a todos los canales suscritos sin acoplar el ticket a los servicios de mensajería.

4. Flujo de Trabajo Profesional con Gitflow

Para registrar los avances de tu diseño y simular un entorno de desarrollo profesional, debes estructurar tus ramas de la siguiente manera:

- main (o master): Código/Documentación en estado de producción (versiones estables de entrega, ej. v1.0.0).
- develop: Tu rama base de integración. Aquí se fusionan los avances del diseño antes de ir a main.
- Ramas feature/ (Características): Creas una rama específica para cada entregable de la actividad. Esto mantendrá tu historial limpio y modular.

5. Evidencias

Secuencia de comandos sugerida para tu repositorio:

5.1 Inicializar y estructura base:

Bash



```
git init
```

```
git checkout -b main
```

```
# Agrega un archivo README.md inicial si lo deseas
```

```
git commit -m "chore: initial commit"
```

```
git checkout -b develop
```

5.2 Trabajar en el Análisis y Casos de Uso (feature/analisis-casos-uso):

Bash

```
git checkout -b feature/analisis-casos-uso develop
```

```
# Creas tus documentos de texto y el diagrama de casos de uso
```

```
git add .
```

```
git commit -m "docs: añadir analisis de requisitos y diagrama de casos de uso UML"
```

```
# Finalizar la característica e integrar a develop
```

```
git checkout develop
```

```
git merge --no-ff feature/analisis-casos-uso
```

```
git branch -d feature/analisis-casos-uso
```

5.3 Trabajar en el Diagrama de Clases y Patrones (feature/diagrama-clases-patrones):

Bash

```
git checkout -b feature/diagrama-clases-patrones develop
```

```
# Diseñas el diagrama de clases integrando Factory, State u Observer
```

```
git add .
```

```
git commit -m "docs: diseñar diagrama de clases incluyendo patrones Factory, State y Observer"
```



```
git checkout develop
```

```
git merge --no-ff feature/diagrama-clases-patronos
```

```
git branch -d feature/diagrama-clases-patronos
```

5.4 Preparar la Entrega Final (release/v1.0.0):

Bash

```
git checkout -b release/v1.0.0 develop
```

```
# Ajustes menores, corrección de formato en la documentación
```

```
git commit -m "docs: preparar entrega final de la Actividad 1"
```

```
# Fusionar a main (Producción)
```

```
git checkout main
```

```
git merge --no-ff release/v1.0.0
```

```
git tag -a v1.0.0 -m "Entrega Actividad 1 - Sistema Help Desk"
```

```
# Actualizar develop
```

```
git checkout develop
```

```
git merge --no-ff release/v1.0.0
```

```
git branch -d release/v1.0.0
```

<https://github.com/maqabre80/DESARROLLOSOFTWAREUTM.git>

https://drive.google.com/file/d/1Slfc7gzqByNx_BFaHN__qCiuY_OAeD-4/view?usp=drive_link



6. Conclusiones

El diseño arquitectónico del sistema de Help Desk para Data Center demuestra cómo la aplicación correcta de patrones de diseño y prácticas de ingeniería de software produce un sistema robusto, escalable y mantenible:

1. El patrón Singleton garantiza la consistencia del estado central del sistema, evitando inconsistencias en un entorno multiusuario y multihilo. Su implementación en el TicketEngine proporciona un único punto de control para toda la lógica de gestión de incidentes.
2. El patrón Observer desacopla el sistema de notificaciones de la lógica de negocio, habilitando una arquitectura reactiva donde los cambios de estado se propagan automáticamente a todos los actores interesados. Esto facilita la adición de nuevos canales de notificación sin modificar el código existente.
3. El patrón Factory Method abstrae la creación de los tres tipos de incidentes (Hardware, Red, Software), permitiendo que el sistema sea extensible ante nuevas categorías. El controlador trabaja con abstracciones en lugar de implementaciones concretas, cumpliendo el principio DIP de SOLID.
4. El patrón MVC estructura toda la aplicación en tres capas independientes, facilitando el mantenimiento, las pruebas unitarias y la posibilidad de cambiar la interfaz de usuario sin afectar la lógica de negocio.
5. La implementación de Gitflow en el repositorio GitHub proporciona trazabilidad completa del historial de desarrollo, con ramas dedicadas para cada funcionalidad y un flujo controlado hacia producción, garantizando la estabilidad del sistema en cada entrega.



La combinación de estos elementos produce una solución que cumple con todos los requerimientos funcionales y no funcionales identificados: escalabilidad horizontal, alta disponibilidad, trazabilidad de versiones y mantenibilidad a largo plazo, siendo una base sólida para el crecimiento futuro del sistema de infraestructura del Data Center.



Bibliografía

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
 - Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
 - Driessen, V. (2010). A successful Git branching model. Nvie.com. <https://nvie.com/posts/a-successful-git-branching-model/>
 - Freeman, E., & Robson, E. (2020). Head First Design Patterns (2nd ed.). O'Reilly Media.
- Object Management Group. (2017). Unified Modeling Language Specification Version 2.5.1. OMG